

Động lực và định nghĩa vấn đề

Mô phỏng, đánh giá, triển khai và phản cải tiến

Mục tiêu bài học

- Xác định động lực và mục tiêu của bài toán.
- Thiết kế **kịch bản thử nghiệm** để thử hệ thống trước khi đưa vào môi trường thật.
- Lựa chọn **chỉ số đánh giá** phù hợp với từng mục tiêu.
- Quyết định **cải tiến, triển khai**, hay tiếp tục mô phỏng/thử nghiệm.
- Nhận diện rủi ro và thực hiện các bước rút kinh nghiệm phù hợp.

MOTIVATION & PROBLEM DEFINITION

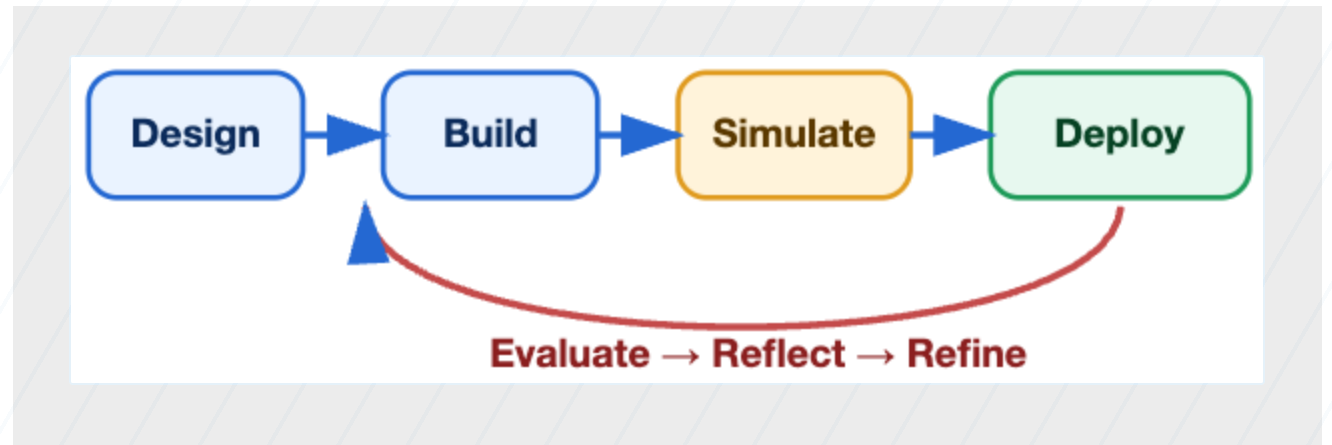
XÁC ĐỊNH ĐÚNG VẤN ĐỀ TRƯỚC KHI ĐÁNH GIÁ VÀ TRIỂN KHAI

Từ thuật toán đến hệ thống thật

Một lời giải tốt không chỉ là lời giải đúng.

Nó còn cần:

- chạy được với dữ liệu thật
- chịu được lỗi và tình huống biên
- đo được hiệu quả
- triển khai được cho người dùng
- cải tiến được sau mỗi vòng phản hồi



Tại sao cần xác định động lực?

Nhiều dự án thất bại không phải vì code sai, mà vì **vấn đề ban đầu không rõ**.

Một vấn đề tốt cần trả lời:

- ❑ **What:** điều gì đang sai, thiếu hoặc gây khó khăn?
- ❑ **Who:** ai là người bị ảnh hưởng?
- ❑ **Where/When:** vấn đề xảy ra trong bối cảnh nào?
- ❑ **Why:** tại sao vấn đề đáng giải quyết?
- ❑ **Constraints:** ràng buộc và kết quả mong đợi là gì?

Khung tham chiếu MEC

Để thuyết phục rằng một vấn đề **đáng giải quyết**, có thể dùng mô hình **MEC**:

.

Magnitude

Vấn đề lớn đến mức nào?

Ví dụ: số người bị ảnh hưởng, số giờ bị mất, số lỗi phát sinh.

Evidence

Bằng chứng nào hỗ trợ?

Ví dụ: thống kê, khảo sát, log hệ thống, phỏng vấn, benchmark.

Consequence

Nếu không giải quyết thì sao?

Ví dụ: chi phí tăng, sai sót, trải nghiệm kém, rủi ro vận hành.

Ví dụ: Motivation cho hệ thống du lịch

Chưa tốt

"Du khách gặp khó khăn khi lên kế hoạch. Chúng tôi muốn xây dựng app để giúp họ."

Vấn đề: quá chung chung, chưa có dữ liệu, chưa rõ ai gặp khó khăn và cải thiện được gì.

Tốt hơn

"Khảo sát 158 du khách thường xuyên cho thấy **72%** dành hơn 3 giờ để so sánh lựa chọn. Thiếu thông tin về visa, lịch trình địa phương và chi phí phát sinh làm tăng sai sót khi lập kế hoạch."

- ❑ **Magnitude:** 72%, hơn 3 giờ
- ❑ **Evidence:** khảo sát + phỏng vấn
- ❑ **Consequence:** mất thời gian, stress, chi phí tăng

Từ Motivation đến Evaluation

Motivation không đứng riêng lẻ; nó quyết định ta mô phỏng gì, đo metric nào, và deploy cho ai.

THÀNH PHẦN	CÂU HỎI CẦN TRẢ LỜI	VÍ DỤ HỆ THỐNG DU LỊCH
Problem	Vấn đề cụ thể là gì?	mất nhiều thời gian so sánh lựa chọn
Stakeholder	Ai bị ảnh hưởng?	du khách tự lên lịch trình
Expected outcome	Cải thiện điều gì?	giảm thời gian lập kế hoạch
Metric	Đo bằng gì?	time-on-task, task success, satisfaction

SIMULATION

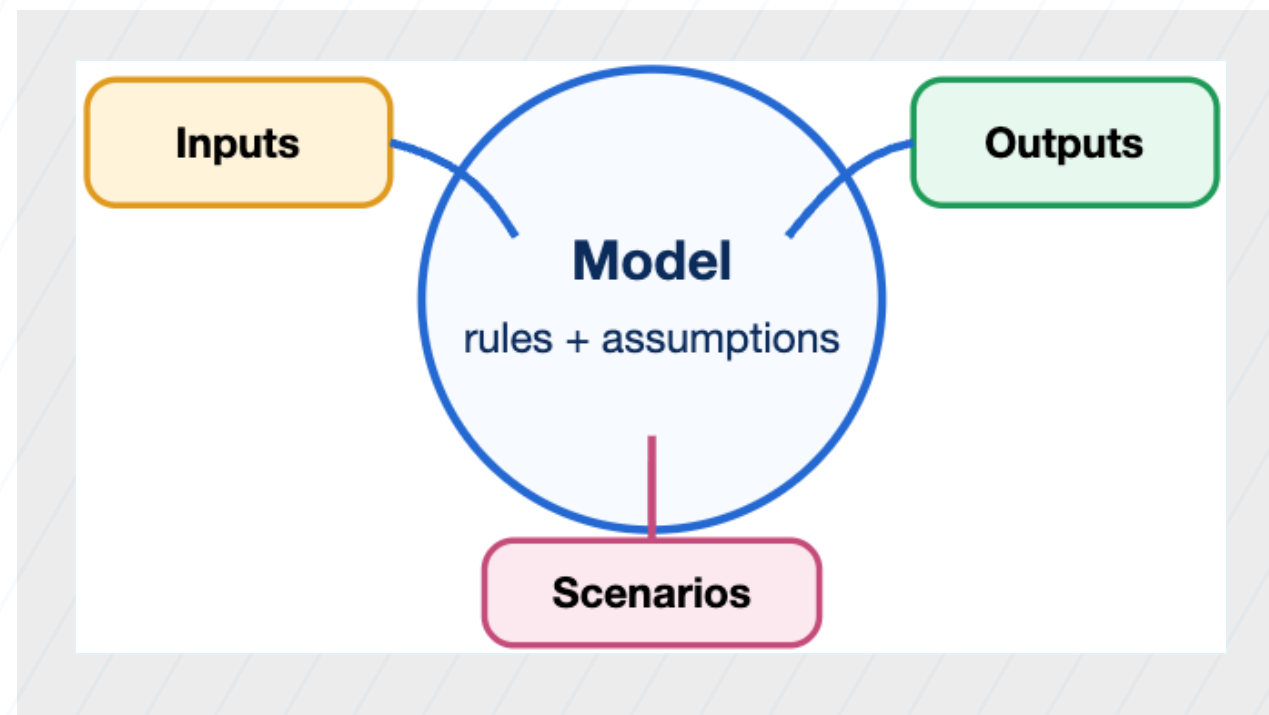
MÔ PHỎNG TRƯỚC KHI THỬ THẬT

Simulation là gì?

Simulation là cách tạo ra môi trường, dữ liệu hoặc kịch bản giả lập để quan sát hệ thống trước khi triển khai rộng.

Mục tiêu

- kiểm tra hành vi trong nhiều tình huống
- phát hiện lỗi khó thấy trong test nhỏ
- giảm chi phí và rủi ro thử nghiệm thật
- hiểu xu hướng khi quy mô tăng



Simulation khác Evaluation thế nào?

NỘI DUNG	SIMULATION	EVALUATION
Câu hỏi chính	"Điều gì có thể xảy ra?"	"Kết quả tốt đến đâu?"
Dữ liệu	giả lập, kịch bản, synthetic, stress	dữ liệu test, benchmark, người dùng thật
Vai trò	thử trước, khám phá rủi ro	đo lường, so sánh, ra quyết định
Kết quả	insight, hành vi, điểm nghẽn	metric, score, trade-off

Quy trình mô phỏng



Ví dụ simulation

Hệ thống đăng ký môn học

Mô phỏng:

- ❑ 1.000 sinh viên đăng ký cùng lúc
- ❑ lớp có giới hạn 60 chỗ
- ❑ một số sinh viên đổi môn liên tục
- ❑ có xung đột lịch học

Cần quan sát

- ❑ hệ thống có bị nghẽn không?
- ❑ waitlist có công bằng không?
- ❑ conflict checking có đúng không?
- ❑ thời gian phản hồi có tăng theo số lượng sinh viên không?

EVALUATION

ĐÁNH GIÁ BẰNG METRIC, SO SÁNH VÀ PHẢN HỒI

Evaluation là gì?

Evaluation là quá trình đo lường và so sánh để trả lời: **hệ thống có tốt không, tốt ở khía cạnh nào, và cần cải thiện gì?**

Các mục tiêu chính

- ❑ **Correctness / Accuracy:** kết quả có đúng không?
- ❑ **Efficiency:** thời gian và tài nguyên có hợp lý không?
- ❑ **Robustness:** có xử lý được lỗi và tình huống biên không?
- ❑ **Scalability:** khi dữ liệu hoặc tải tăng thì còn chạy tốt không?
- ❑ **Usability / Practicality:** có dễ dùng và triển khai thực tế không?

Nguyên tắc đánh giá

1. Measurement

Dùng số đo cụ thể: runtime, memory, accuracy, error rate, cost...

2. Comparison

So với yêu cầu, baseline, phiên bản trước hoặc best practice.

3. Feedback

Dùng kết quả để refine thuật toán, dữ liệu hoặc decomposition.

Simulate → Evaluate → Refine → Deploy → Monitor

Các mô hình đánh giá

User study

Tuyển người dùng thử, phỏng vấn, quan sát lý do họ chọn/hành động.

Online evaluation

A/B testing trên hệ thống đã deploy; so sánh click, conversion, purchase...

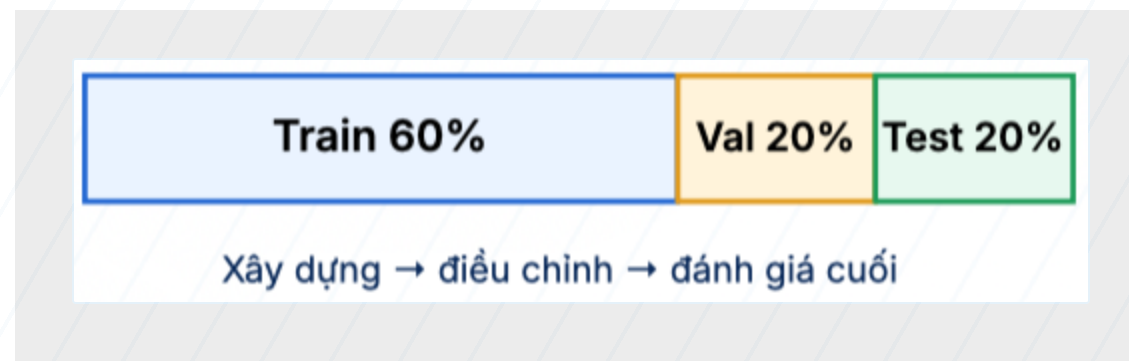
Offline evaluation

Dùng dữ liệu lịch sử/benchmark; rẻ hơn và phù hợp để tuning nhiều lần.

Data splitting cho offline evaluation

- **Training data:** xây dựng model
- **Validation data:** chọn tham số, chọn mô hình
- **Testing data:** đánh giá cuối cùng

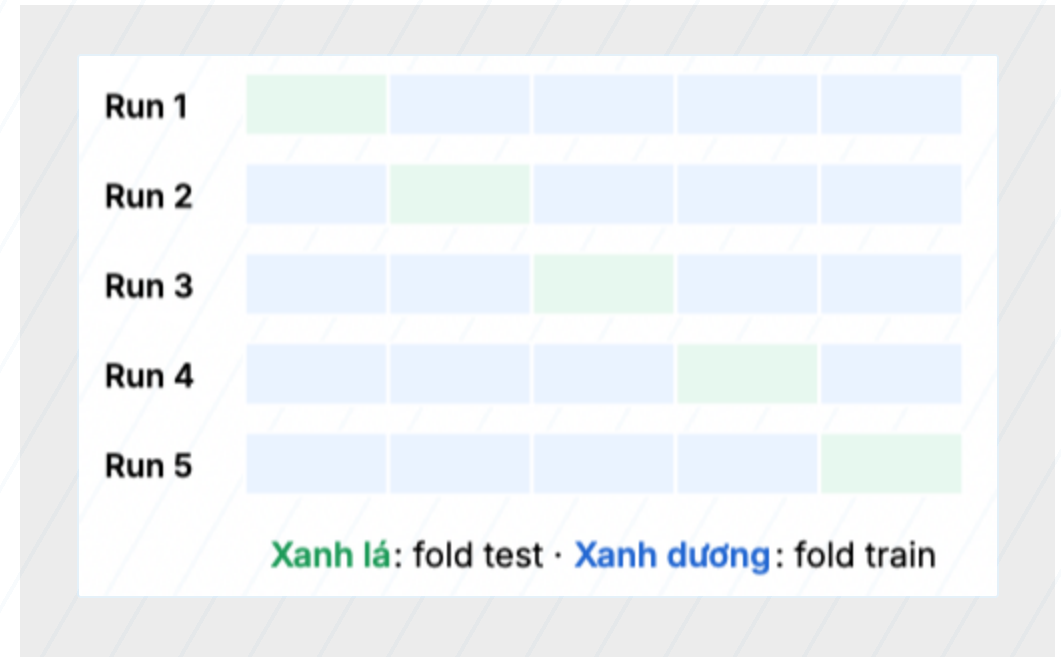
Không dùng testing data cho bất kỳ bước tuning nào.



Cross-validation

K-fold

- Chia dữ liệu thành k phần
- Mỗi lần chọn 1 phần làm test
- Các phần còn lại dùng để train/validation
- Lặp k lần và lấy kết quả trung bình



Chọn metric theo loại bài toán

BÀI TOÁN	OUTPUT	METRIC NÊN DÙNG
Classification	nhãn đúng/sai	Accuracy, Precision, Recall, F1
System performance	dịch vụ/API	latency, throughput, error rate, cost
Usability	trải nghiệm người dùng	task success, time-on-task, satisfaction
Regression / Rating	giá trị số	MAE, RMSE
Ranking / Recommendation	danh sách đã sắp hạng	MAP, MRR, NDCG, AUC

Evaluation: System performance

Dùng khi đánh giá **dịch vụ/API**, hệ thống backend hoặc ứng dụng cần phục vụ nhiều người dùng.

METRIC	Ý NGHĨA	CÁCH ĐỌC NHANH
Latency	thời gian phản hồi	càng thấp càng tốt; nên xem p50/p95/p99
Throughput	số request xử lý mỗi giây	càng cao càng tốt nếu vẫn ổn định
Error rate	tỷ lệ request lỗi	lỗi 4xx/5xx, timeout, failed jobs
Cost	chi phí vận hành	chi phí / request, / user, / task

Evaluation: Usability

Dùng khi đánh giá **trải nghiệm người dùng**: hệ thống có giúp người dùng hoàn thành việc cần làm một cách dễ dàng không?

METRIC	ĐO ĐIỀU GÌ?	VÍ DỤ
Task success	người dùng hoàn thành nhiệm vụ không	% đặt vé thành công
Time-on-task	mất bao lâu để hoàn thành	số phút tạo lịch trình
Satisfaction	mức hài lòng sau khi dùng	rating 1–5, SUS, feedback

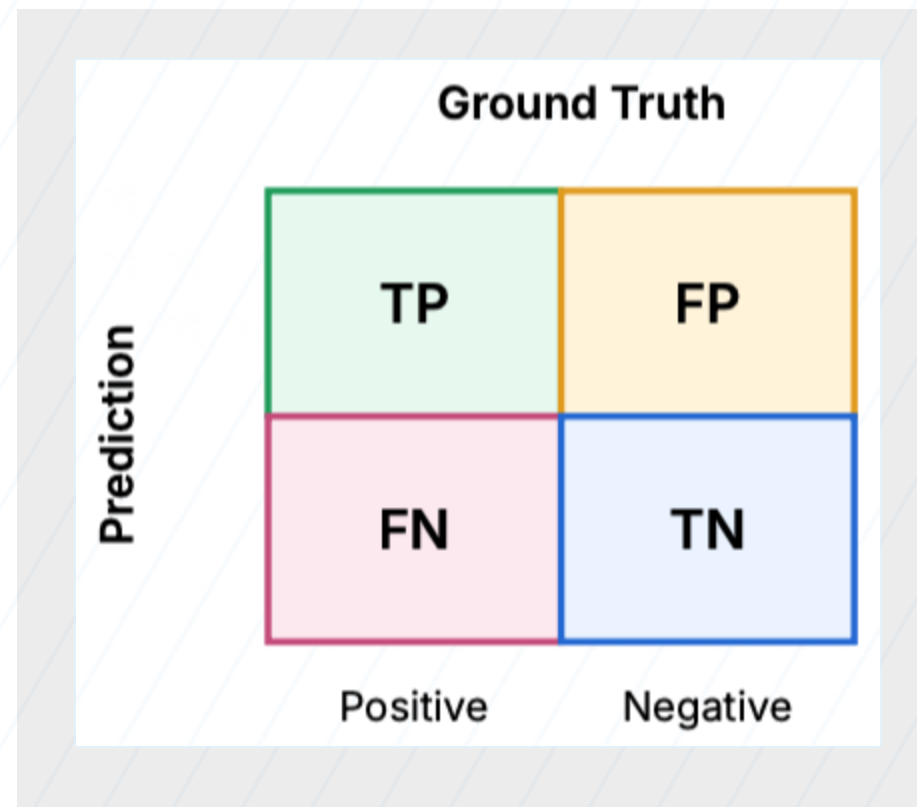
Evaluation: Usability (2)

Khi dùng

- demo sản phẩm, app, chatbot, website
- so sánh phiên bản UI A/B
- kiểm tra xem giải pháp có thực sự hữu ích không

Classification: Precision, Recall, F1

- **Precision:** trong các mẫu được dự đoán positive, bao nhiêu mẫu thật sự positive?
- **Recall:** trong các mẫu thật sự positive, mô hình tìm được bao nhiêu?
- **F1-score:** cân bằng giữa precision và recall.



Regression: MAE và RMSE

MAE

$$MAE = \frac{1}{n} \sum |y_i - \hat{y}_i|$$

- dễ hiểu
- ít bị outlier chi phối hơn

RMSE

$$RMSE = \sqrt{\frac{1}{n} \sum (y_i - \hat{y}_i)^2}$$

- phạt lỗi lớn mạnh hơn
- phù hợp khi lỗi lớn rất nguy hiểm

Ranking metrics: khi thứ tự quan trọng

Ví dụ: hệ thống gợi ý phim, sản phẩm, bài viết.

Không chỉ hỏi: “có gợi ý đúng không?”

Mà còn hỏi: “mục đúng có nằm ở vị trí cao không?”

METRIC	TẬP TRUNG VÀO
MRR	item đúng đầu tiên
MAP	precision tại các điểm recall
NDCG	điểm liên quan và vị trí
AUC	thứ tự positive trước negative

Lỗi thường gặp khi evaluation

- ❑ Chỉ nhìn Big-O, bỏ qua constant factor và dữ liệu thật
- ❑ Chỉ đo runtime, bỏ qua memory, I/O, cost
- ❑ Test trên dữ liệu không đại diện
- ❑ Hy sinh correctness để lấy speed
- ❑ Benchmark quá ít lần, kết quả nhiễu
- ❑ Tối ưu một metric duy nhất
- ❑ Không đánh giá scalability và vận hành
- ❑ Nhận xét định tính nhưng không có số đo
- ❑ Confirmation bias: chỉ tìm bằng chứng ủng hộ lựa chọn sẵn có
- ❑ Đánh giá một lần rồi không đánh giá lại

Evaluation checklist

- 01** Kết quả có đúng trên mọi input hợp lệ không?
- 02** Đã test cả dữ liệu thường, dữ liệu biên và dữ liệu xấu chưa?
- 03** Benchmark có lặp lại đủ nhiều lần không?
- 04** Đã đo runtime, memory, I/O, chi phí và khả năng mở rộng chưa?
- 05** Metric có phù hợp với mục tiêu nghiệp vụ/ngữ cảnh thực tế không?
- 06** Có kế hoạch đánh giá lại khi dữ liệu, thư viện hoặc môi trường thay đổi không?

DEPLOYMENT

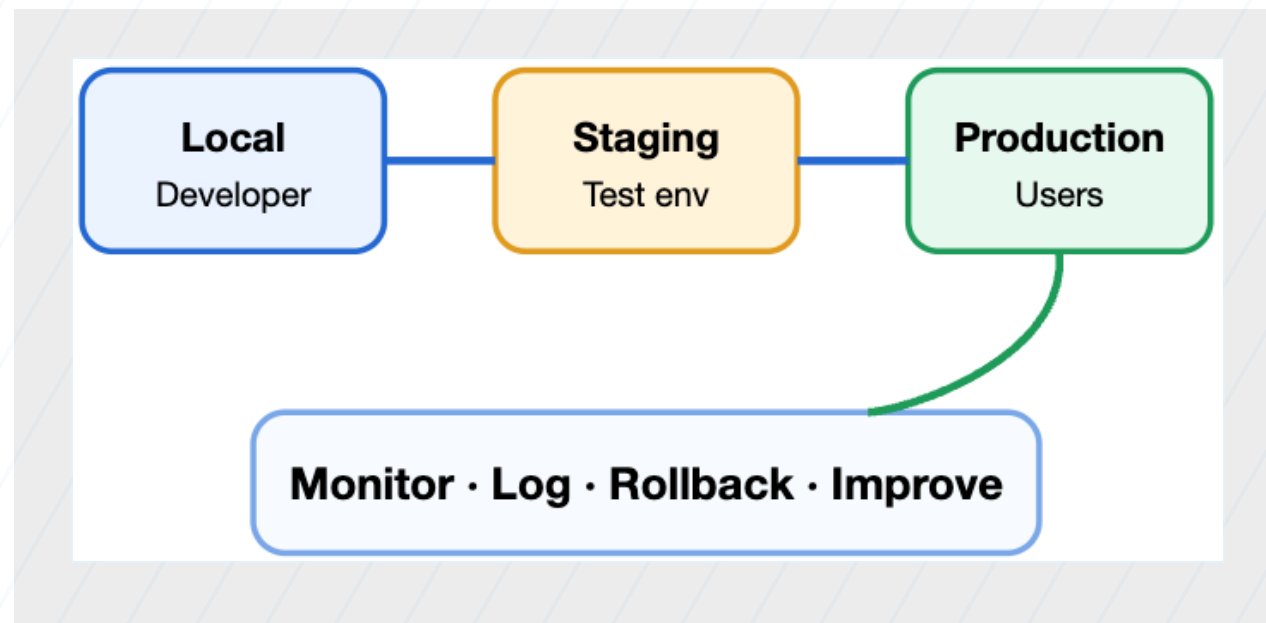
ĐƯA GIẢI PHÁP VÀO MÔI TRƯỜNG THỰC TẾ

Deployment là gì?

Deployment là quá trình chuyển lời giải từ môi trường phát triển sang môi trường người dùng có thể sử dụng.

Không chỉ là “copy code lên server”

- ❑ đóng gói chương trình
- ❑ cấu hình môi trường
- ❑ kết nối dữ liệu/API
- ❑ kiểm thử trước khi phát hành
- ❑ giám sát sau khi phát hành
- ❑ có kế hoạch rollback khi lỗi



Deployment checklist

Trước khi deploy

- ❑ Chạy test case quan trọng
- ❑ Kiểm tra dữ liệu đầu vào/đầu ra
- ❑ Cố định phiên bản thư viện
- ❑ Cấu hình biến môi trường
- ❑ Có dữ liệu mẫu để kiểm tra nhanh

Sau khi deploy

- ❑ Theo dõi log và lỗi
- ❑ Đo latency, memory, CPU, cost
- ❑ Thu thập feedback người dùng
- ❑ So sánh với phiên bản trước
- ❑ Chuẩn bị rollback nếu phát hiện lỗi nghiêm trọng

Các kiểu triển khai thường gặp

Kiểu Deploy	Ý nghĩa	Khi nên dùng
Manual	Tự chạy lệnh, tự copy file	Bài tập nhỏ, demo nhanh
Scripted	Có script lặp lại được	Dự án nhóm, giảm lỗi thao tác
CI/CD	Tự test, build, release	Sản phẩm cập nhật thường xuyên
Canary / A-B	Mở dần cho một nhóm người dùng	Muốn giảm rủi ro khi thay đổi lớn

Rủi ro khi deploy

- ❑ Code đúng trên máy cá nhân nhưng sai ở server
- ❑ Dữ liệu thật khác dữ liệu test
- ❑ Thiếu quyền truy cập file/API/database
- ❑ Phiên bản thư viện không giống nhau
- ❑ Người dùng thao tác ngoài dự kiến

Quy tắc an toàn

Deploy càng gần thực tế thì càng phải đo, ghi log và có đường quay lại.

REFLECTION

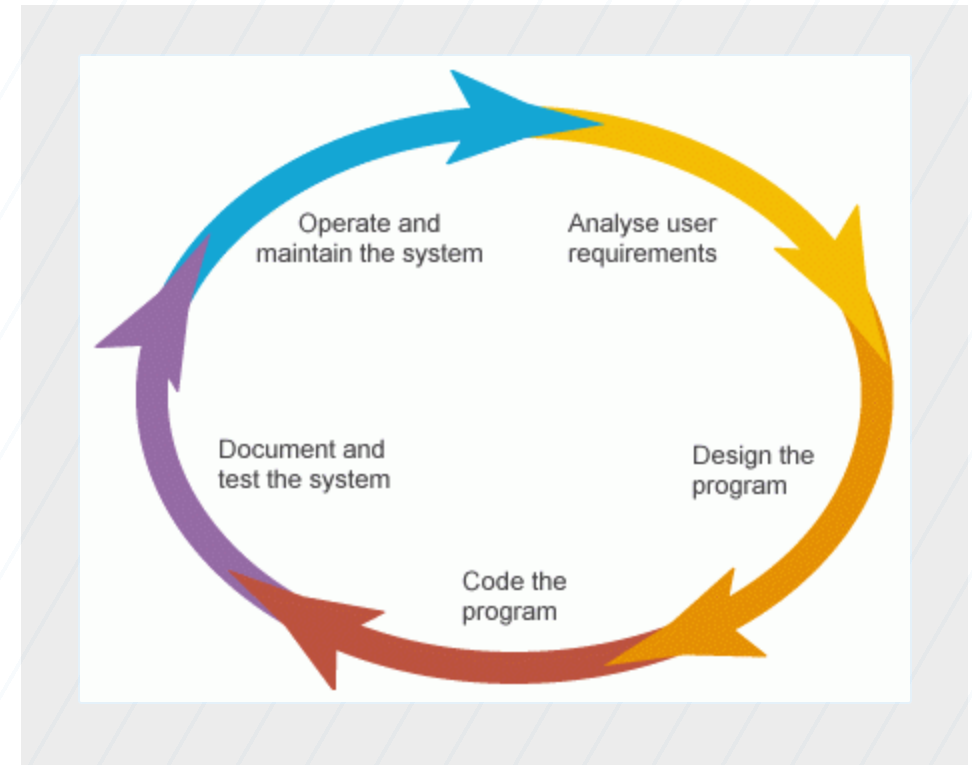
NGHĨ LẠI ĐỂ HỌC VÀ CẢI TIẾN

Reflection là gì?

Reflection là quá trình nhìn lại toàn bộ cách ta giải quyết vấn đề để rút ra bài học và cải tiến chiến lược.

Mục đích

- nhận diện điểm mạnh và điểm yếu
- hiểu vì sao một quyết định đúng/sai
- biến sai lầm thành checklist
- phát triển kỹ năng tư duy tính toán dài hạn



Nguyên tắc reflection

Self-assessment

Nhìn lại toàn bộ quá trình:
decomposition,
abstraction, coding,
testing, debugging.

Feedback integration

Dùng kết quả evaluation để
sửa thiết kế, dữ liệu hoặc
cách triển khai.

Meta-cognition

Ý thức được cách mình suy
nghĩ, sai ở đâu, và pattern
nào có thể tái sử dụng.

Phương pháp reflection

Review / Retrospective

So sánh giải pháp với yêu cầu ban đầu; xem lại quyết định thuật toán và biểu diễn dữ liệu.

Error analysis

Tìm nguyên nhân gốc của lỗi: phân rã sai, giả định sai, test thiếu, abstraction không phù hợp.

Performance reflection

So sánh runtime, memory, complexity và trade-off.

Strategy reflection

Hỏi: có cách đơn giản hơn không? pattern nào có thể tái sử dụng?

Journaling

Ghi lại lesson learned để dùng cho bài sau.

Lỗi thường gặp khi reflection

- ❑ Bỏ qua reflection sau khi nộp bài
- ❑ Chỉ viết "it works" mà không phân tích vì sao
- ❑ Chỉ nhìn correctness, bỏ qua quá trình
- ❑ Không ghi learning log
- ❑ Không xem lời giải hoặc cách làm của người khác
- ❑ Reflection quá mơ hồ: "code hơi chậm" nhưng không có số đo
- ❑ Chỉ nhớ điều mình làm đúng, bỏ qua lỗi lớn
- ❑ Phản tư một lần duy nhất rồi quên
- ❑ Không biến bài học thành hành động cụ thể
- ❑ Nhầm reflection với tự trách tiêu cực

Reflection checklist

- 01** Tôi đã dành bao nhiêu thời gian cho decomposition, design, coding, debugging?
- 02** Sai lầm nghiêm trọng nhất là gì?
- 03** Nếu làm lại, tôi sẽ đổi chiến lược/abstraction/implementation nào?
- 04** Tôi có bỏ lỡ lời giải đơn giản hoặc đẹp hơn không?
- 05** Bài học cụ thể là gì?
- 06** Tuần sau tôi sẽ luyện kỹ năng nào dựa trên bài học này?



CASE STUDY

FAKE NEWS DETECTION / RECOMMENDATION SYSTEM

Case study 1: Fake News Detection

Quy trình chính

- ❑ Đọc dữ liệu và thống kê cơ bản
- ❑ Clean text: lowercase, stop words, stemming...
- ❑ Split train/validation/test
- ❑ Vectorize text
- ❑ Train model
- ❑ Evaluate bằng confusion matrix, precision, recall, F1

Reflection sau đánh giá

- ❑ Dữ liệu có lệch nhãn không?
- ❑ Mô hình sai nhiều ở loại tin nào?
- ❑ Precision hay recall quan trọng hơn?
- ❑ Có overfit vào từ khóa bề mặt không?
- ❑ Cần thêm dữ liệu hay đổi representation?

Case study: motivate + simulate + evaluate + deploy + reflect

GIAI ĐOẠN	FAKE NEWS DETECTION	RECOMMENDATION SYSTEM
Motivate	phát hiện tin giả để giảm sai lệch thông tin	gợi ý đúng nội dung để tăng giá trị ngu
Simulate	tin nhiều, tin ngắn, tin trùng	user mới, item mới, tải lớn
Evaluate	Precision, Recall, F1	MAP, MRR, NDCG, AUC, latency
Deploy	API phân loại tin	API gợi ý Top-N

CONTINUOUS IMPROVEMENT

DUY TRÌ ĐỘNG LỰC CẢI TIẾN

Motivation trong vòng cải tiến

Động lực không chỉ đến từ điểm số hay việc "chạy được".

Động lực bền vững đến từ việc thấy rõ:

- ❑ mình đã cải thiện metric nào
- ❑ lỗi cũ có còn lặp lại không
- ❑ pattern nào đã học được
- ❑ giải pháp có tạo giá trị thật không
- ❑ vòng sau có mục tiêu nhỏ và đo được không

Bài tập trên lớp

Chọn một hệ thống và thiết kế vòng cải tiến

Ví dụ: đăng ký môn học, phân loại email spam, tìm đường đi, gợi ý phim, chatbot hỗ trợ sinh viên....Cần trả lời:

- 01** Motivation: vấn đề nào đáng giải quyết?
- 02** Simulation cần những kịch bản nào?
- 03** Evaluation dùng metric nào?
- 04** Deploy cho ai dùng trước và dùng ở đâu?
- 05** Reflection hỏi lại điều gì?
- 06** Mục tiêu cải tiến nhỏ nhất cho vòng sau là gì?

Tổng kết

- **Motivation:** xác định vấn đề đáng giải quyết và mục tiêu đo được.
- **Simulation:** thử nhiều kịch bản trước khi rủi ro xảy ra thật.
- **Evaluation:** đo lường bằng metric, so sánh với baseline và yêu cầu.
- **Deployment:** đưa giải pháp vào môi trường thật khi đã đủ điều kiện.
- **Reflection:** nhìn lại quá trình để rút bài học và tạo vòng cải tiến tiếp theo.

Bài tập về nhà

Evaluation & Reflection Report

Chọn một bài tập lập trình hoặc mini-project đã làm.

Nộp báo cáo 1–2 trang gồm:

- 01** Motivation và mục tiêu hệ thống
- 02** Kịch bản simulation
- 03** Metric evaluation và kết quả
- 04** Cách deploy/demo
- 05** Reflection checklist
- 06** Một action item cụ thể cho lần cải tiến tiếp theo

Tài liệu tham khảo

- Slide bài giảng **CT13.Evaluation.**
- Slide bài giảng **CT14.Reflection.**
- Slide bài giảng **CT_Week09_Evaluation.**
- Nội dung Marp template hiện có của học phần CSC10014.

Q&A